

ILP and CP Formulations for the Lazy Bureaucrat Problem

Fabio Furini¹ Ivana Ljubić²
Markus Sinnl²

PSL, Université Paris-Dauphine, 75775 Paris Cedex 16, France,
CNRS, LAMSADE UMR 7243
`fabio.furini@dauphine.fr`

Department of Statistics and Operations Research,
University of Vienna, Vienna, Austria
`{ivana.ljubic,markus.sinnl}@univie.ac.at`

October 3, 2026

Abstract

Lazy reformulations of classical combinatorial optimization problems are new and challenging classes of problems. In this paper we focus on the Lazy Bureaucrat Problem (LBP) which is the lazy counterpart of the knapsack problem. Given a set of tasks with a common arrival time and deadline, the goal of a lazy bureaucrat is to schedule a *least* profitable subset of tasks, while having an excuse that no other tasks can be scheduled without exceeding the deadline.

Three ILP formulations and their CP counterparts are studied and implemented. In addition, a dynamic programming algorithm that runs in pseudo-polynomial time and polynomial greedy heuristics are implemented and computationally compared with ILP/CP approaches. For the computational study, a large set of knapsack-type instances with various characteristics is used to examine the applicability and strength of the proposed approaches.

1. Introduction

The lazy bureaucrat problem is a scheduling problem in which a set of jobs has to be scheduled in a most inefficient way. We will consider a special variant of this problem in which all jobs are arriving at the same time and have a common deadline. In this paper, we will simply refer to this problem as the Lazy Bureaucrat Problem (LBP).

The problem is motivated by the following application: a lazy bureaucrat needs to choose a subset of jobs to execute in a single day, in a such a way that no other job fits in his/her working hours and the total profit of selected jobs (e.g., their duration) is minimized (and, hence, for example, the bureaucrat has a good excuse to go home as early as possible). Governments or funding agencies may be interested in applications of the LBP: how to distribute the available

budget so that the minimal amount of money is allocated to funding requests, while having a good excuse that no additional funds can be granted without violating the available budget? For further applications and motivations of this problem, see, e.g., (Arkin et al., 2003; Gourv  s et al., 2013). The study of the LBP could potentially lead to new interesting insights for knapsack-type problems, see also the conclusion for a discussion on the importance of the LBP.

Definition 1.1 (Lazy Bureaucrat Problem (with common arrivals and deadlines), LBP). We are given a set of jobs $I = \{1, \dots, n\}$, such that each job $i \in I$ is assigned a duration $w_i \in \mathbb{N}$ and a profit $p_i \in \mathbb{N}$. All jobs arrive at the same time, and all have a common deadline $C \in \mathbb{N}$. The goal is to find a least profitable subset of jobs S to be executed so that the schedule cannot be improved by inserting an additional job into it. More precisely, the optimal solution $S^* \subset I$ solves the following problem:

$$S^* = \arg \min_{S \subset I} \left\{ \sum_{i \in S} p_i \mid \sum_{i \in S} w_i \leq C \text{ and } \sum_{i \in S} w_i + w_j > C, \forall j \notin S \right\}.$$

Note that we assume that the problem instance is non-trivial, so that the optimal solution S is a proper subset of I . The objective considered in this paper is of a very general form and referred to in literature as **weighted-sum**. It generalises the **min-number-of-jobs** objective, which is obtained for $p_i = 1$ and the **time-spent** objective, which is obtained for $p_i = w_i$.

The problem has been introduced in Arkin et al. (2003) where it was shown that a more general problem variant with individual arrival times and deadlines is NP-hard. For the problem variant with common arrival times and deadlines, Gai and Zhang (2008) show that the problem is weakly NP-hard for the **min-number-of-jobs** objective by reduction from subset-sum. Thus, the problem studied in this paper (with the more general **weighted-sum** objective) is also at least weakly NP-hard. Note that in Arkin et al. (2003) it is claimed, that the LBP with common arrival times can be solved in pseudo-polynomial time for various objective functions (including **time-spent** and **weighted-sum**), but neither proof nor the corresponding algorithm were explicitly given for this claim. In Gourv  s et al. (2013) two greedy heuristics and an FPTAS have been proposed for the **time-spent** objective with common arrival and deadlines. The FPTAS is obtained as an approximate version of an exact enumeration algorithm.

Our Contribution: In this article we provide a first dynamic programming algorithm for the **weighted-sum** LBP (see Section 2) that runs in pseudo-polynomial time. Thus the problem is indeed weakly NP-hard. Besides, we prove properties of optimal solutions that can be used to derive more efficient mathematical models and algorithmic approaches or to derive valid inequalities or stronger constraints in both integer linear programming (ILP) and constraint programming (CP) formulations. Three ILP models including their CP counterparts are presented in Section 3 and greedy heuristics are discussed in Section 4. An extensive computational study is conducted on a large set of diverse benchmark instances, demonstrating the strengths and weaknesses of the proposed approaches.

Notation: Notice that the LBP consists of selecting a subset of jobs, but due to the common arrival times and the common deadline, the order in which the jobs are scheduled remains irrelevant. Therefore, the LBP shares a lot of similarities with the knapsack problem (KP), and can be seen as the problem of packing a set of items in a knapsack in a most inefficient (and therefore, rather counterintuitive) way. Due to this similarity to the knapsack problem, in the remainder of this article we will refer to elements of I as *items*, job durations w_i as *item weights* and p_i as *item profits*. The deadline C will be called the *budget* or *capacity*.

In the following, we will assume that all items are sorted in non-decreasing lexicographic order, first according to their weight, i.e., $w_1 \leq w_2 \leq \dots \leq w_n$, and then, according to their profits p_i . For each $i \in I$, let $C_i := C - w_i$, so we have $C_1 \geq C_2 \geq \dots \geq C_n$. We will also denote by $w_{\max} := \max_{i \in I} w_i (= w_n)$ and $w_{\min} := \min_{i \in I} w_i (= w_1)$. Let $W := \sum_{i \in I} w_i$ and $P := \sum_{i \in I} p_i$.

1.1 Solution Properties

In this section we point out some general solution properties that will be used to prove the validity of our models.

Property 1.2. *The capacity used by an arbitrary feasible solution S is bounded from below by $C - w_{\max} + 1$.*

Proof. By definition, inserting any item outside of S into the knapsack will exceed its capacity. So, in particular, the capacity of S plus $w_{\max}$ must be $\geq C + 1$, i.e., the capacity of S must be at least $C - w_{\max} + 1$. Since $w_{\max} \geq w_i, \forall i \in I$, this bound is clearly valid for any item. $\square \square$

Despite the minimization objective, imposing the upper bound on the size of the knapsack is in general not redundant. Consider the following example in which we are given three items such that $w_1 = 1$, $w_2 = 2$ and $w_3 = 3$, and $p_1 = 10$, $p_2 = p_3 = 1$, and that the knapsack capacity is $C = 4$. Without imposing the knapsack capacity, the optimal solution will be to take the items $\{2, 3\}$ with the total profit of 2, whereas, with the condition of not exceeding the knapsack capacity, the optimal solution will be $\{1, 3\}$ with the total profit of 11. However, in the **time-spent** problem variant, the knapsack constraint is redundant, as shown by the following result:

Property 1.3. *If $w_i = p_i$ for all $i \in I$, capacity of any optimal solution S will not exceed C , even without explicitly imposing this condition.*

Definition 1.4 (Minimal Knapsack Cover). Given a knapsack problem with capacity C , a cover $\mathcal{C}$ is a set of items, with the property $\sum_{c \in \mathcal{C}} w_c \geq C + 1$, i.e., a subset of items, which exceeds the capacity C . A cover $\mathcal{C}$ is called *minimal*, iff $\mathcal{C} \setminus \{i\}$ is not a cover for any $i \in \mathcal{C}$. We denote the family of all minimal covers with $\mathcal{K}$.

Definition 1.5 (Maximal Knapsack Packing). Given a knapsack problem with capacity C , a packing $\mathcal{P}$ is a set of items, with the property $\sum_{i \in \mathcal{P}} w_i \leq C$, i.e., a subset of items, which does not exceed the capacity C . A packing $\mathcal{P}$ is called *maximal*, iff $\mathcal{P} \cup \{i\}$ is not a packing for any $i \notin \mathcal{P}$.

The following property characterizes the set of feasible LBP solutions.

Property 1.6. *Each feasible LBP solution corresponds to a maximal feasible packing of the knapsack with capacity C .*

It is worth mentioning that searching for a minimum-profit minimal knapsack cover, and removing an item from it does not lead to an optimal solution of the LBP. Even in the simplest case when $p_i = w_i$, for all $i \in I$, this does not hold: Let the set of items be ordered according to their weights(=profits) $\{31, 32, 33, 40, 45\}$ and let the knapsack capacity be 80. The minimum-profit minimal knapsack cover is $\{40, 45\}$, and after removing one of these items from the cover, the obtained solution is not feasible to LBP (since one can obviously add one of the items from $\{31, 32, 33\}$ without violating the knapsack capacity).

Definition 1.7 (Critical Weight and Critical Item.). Assume that items are sorted so that $w_1 \leq w_2 \leq \dots \leq w_n$. Denote by

$$i_c = \arg \min \{i \in I \mid \sum_{j \leq i} w_j > C\}$$

the index of a *critical item*, i.e., the index of the first item that exceeds the capacity, assuming all $i \leq i_c$ will be taken as well. The *critical weight*, denoted by w_c , is the weight of the critical item, i.e., $w_c = w_{i_c}$.

Note that the critical item and its weight are uniquely defined, assuming the lexicographic ordering of the items defined above.

Proposition 1.8. *The weight of the smallest item left out of any feasible LBP solution is bounded above by the critical weight w_c , i.e.:*

$$S \text{ is feasible} \Rightarrow \min_{i \notin S} w_i \leq w_c.$$

Consequently, the size of the knapsack can be bounded from below as:

$$w(S) \geq C - w_c + 1.$$

Proof. Suppose there exists a feasible solution S' with $\min_{i \notin S'} w_i > w_c$. Since the items are ordered by weight, it follows that all items $i \leq i_c$ are in S' . By the definition of the critical item, the sum of the weights of these items exceeds C and thus S' is not feasible. The second part is a refinement of Property 1.2. □ □

2. Dynamic Programming Approach

The latter result indicates a possible iterative approach for solving the LBP. Once, the smallest item left out of the solution is known, the problem reduces to solving the knapsack problem with a lower and upper bound on its capacity (which we will denote LU-KP below). Let S^*

be an optimal LBP solution and assume that $i \leq i_c$ is the index of the smallest item not taken into S^* (following the lexicographic ordering of items). Let I_i be the set of the first $i - 1$ items, $I_i = \{1, \dots, i - 1\}$ ($I_1 = \emptyset$), and let $P_i = \sum_{j < i} p_j$ and $W_i = \sum_{j < i} w_j$ be the profits and weights of items from I_i . Then, $S^* = I_i \cup J^*$ where $J^* \subset \{i + 1, \dots, n\}$ solves the following problem:

$$(\text{KP}_i^{\min})J^* = \arg \min_{J \subseteq \{i+1, \dots, n\}} \left\{ \sum_{j \in J} p_j + P_i \mid C - w_i - W_i + 1 \leq \sum_{j \in J} w_j \leq C - W_i \right\}$$

The optimal solution of $(\text{KP}_i^{\min})$ can be obtained by solving the complementary maximization problem, which is a LU-KP, as shown in the following result:

Proposition 2.1. *The optimal LBP solution S^* can be obtained as:*

$$i^* = \arg \min_{i \leq i_c} \max(\text{KP}_i) \text{ and } S^* = I \setminus \arg \max(\text{KP}_{i^*})$$

where $\max(\text{KP}_i)$ is defined as:

$$\max(\text{KP}_i) = \max \left\{ \sum_{j > i} p_j y_j \mid \sum_{j > i} w_j y_j \geq C'_i, \sum_{j > i} w_j y_j \leq C''_i, y_j \in \{0, 1\}, \forall j > i \right\},$$

with $C'_i := W - C - w_i$ and $C''_i := W - C + 1$.

Proof. Let J^* be defined as above and let the binary variables y_j be equal to one if $j \notin J^*$ and to zero, otherwise. By rewriting the problem $(\text{KP}_i^{\min})$ stated above as a maximization problem and using y_i variables to indicate items outside of the solution, we end up with the desired reformulation. The value $\text{KP}_{i^*}^{\min}$ is obtained as $P - \text{KP}_{i^*}$. $\square$ $\square$

Proposition 2.1 demonstrates that the LBP could be solved in i_c iterations, in each of which a LU-KP is solved to optimality to obtain $\max(\text{KP}_i)$ for an $i \leq i_c$. This can be done in $O(n^2W)$ time, by applying a standard dynamic programming procedure for the LU-KP that runs in $O(nW)$. We demonstrate below a modification of this algorithm that is based on iteratively solving $\text{KP}_i^{\min}$ by dynamic programming. Similar iterative arguments are used to construct the exact enumeration algorithm underlying the FPTAS for the **time-spent** variant in (Gourv  s et al., 2013).

Let $C_i^l := C - w_i - W_i + 1$ and $C_i^u = C - W_i$. For solving $\text{KP}_i^{\min}$ (for a fixed i) we need to apply a “dynamic programming by weights” approach, in which the smallest profit $z(j)$ that can be obtained for each capacity $j \in \{1, \dots, C_i^u\}$ is computed. The following recursive formulas are used (see, e.g., Casazza and Ceselli (2014)):

$$\begin{cases} M(0, 0) = 0, & M(0, j) = -\infty, & \forall j \leq C_i^u \\ M(i', j) = \max\{M(i' - 1, j), M(i' - 1, j - w_{i'}) - p_{i'}\}, & i' > i, \forall j \in \{1, \dots, C_i^u\} \end{cases}$$

The algorithm searches for the filling of the knapsack of capacity j , $j \in \{0, 1, \dots, C_i^u\}$, that maximizes the sum of negative item profits (which is equivalent to minimizing the sum of the

profit of collected items). That way, only the cells containing values $\neq -\infty$ represent feasible solutions, whose total weight is exactly equal to j , i.e., cell $M(i', j)$ contains the maximum possible sum of negative item profits which can be achieved by a solution with weight j using items up to i' . Let $z(j)$ denote the complemented value at the cell $M(|I|, j)$ after applying the DP. The optimal solution of $KP_i^{\min}$ is obtained as $\min_{C_i^l \leq j \leq C_i^u} z(j)$.

3. ILP and CP Formulations

The DP presented above relies on solving i_c iterations of the LU-KP, and hence the approach may become prohibitive from the computational perspective if C and/or i_c are very large. In this section we study alternative, *integrative* approaches, in which the LBP is solved as a whole. We propose three ways to formulate the problem, always presenting the valid ILP formulation first, followed by its CP counterpart. We provide some valid inequalities and show how some of them can be lifted to obtain stronger lower bounds. A hybridization of the proposed ILPs and CPs is also discussed in this section as well as a branch-and-cut algorithm based on the ILPs.

3.1 A First Formulation

The first ILP model is obtained by formulating the problem using the binary variables x_i which are set to one if the item is selected, and to zero, otherwise. Each feasible solutions needs to fit into a knapsack, and should exceed its capacity by adding an arbitrary additional item left outside. The formulation uses the Proposition 1.8, and considers only the items left outside of S whose weight does not exceed the critical weight. The model reads as follows:

$$\begin{aligned}
 (\text{ILP}_1) \quad & \min_{x \in \{0,1\}^{|I|}} \sum_{i \in I} p_i x_i \\
 & \sum_{i \in I} w_i x_i \leq C \tag{1}
 \end{aligned}$$

$$\sum_{j \in I, j \neq i} w_j x_j + w_i(1 - x_i) \geq (C + 1)(1 - x_i) \quad \forall i \in I : i \leq i_c \tag{2}$$

Constraint (1) is a knapsack constraint stating that the weight of all selected items cannot exceed the available budget C . Inequalities (2) make sure that adding each additional item i such that $w_i \leq w_c$ will exceed C . We will refer to them as *covering inequalities* associated to items $i \leq i_c$. Notice that the latter inequalities can be rewritten as: $\sum_{j \in I, j \neq i} w_j x_j \geq (C_i + 1)(1 - x_i) \quad \forall i \in I : i \leq i_c$.

Proposition 3.1. *The model (ILP₁) is a valid formulation for the LBP.*

3.1.1 Strengthening Covering Inequalities

Proposition 3.1 shows that it is sufficient to consider items not exceeding the critical weight to enforce a feasible LBP solution. It is not difficult to see that covering inequalities associated

with remaining items $i > i_c$ are also valid for our problem:

$$\sum_{j \in I, j \neq i} w_j x_j + w_i(1 - x_i) \geq (C + 1)(1 - x_i) \quad \forall i \in I : i > i_c$$

However, all these constraints are dominated by the *global covering constraint*, derived from the global lower bound given in Proposition 1.8:

$$\sum_{j \in I} w_j x_j \geq C_c + 1, \quad (3)$$

where $C_c = C - w_c$. One can easily construct an example showing that the lower bound of the LP relaxation of the model (ILP₁) is strengthened by adding this constraint: Let the item weights be $w = p = (1, 119, 552, 605, 739, 863)^t$. The optimal LP-solution of the (ILP₁) is $x' = (0.4685, 0.4647, 0.4357, 0.4285, 0.4004, 0.3435)^t$ with the LP-relaxation value equal to 1147.91. However, the optimal LP-solution of the (ILP₁) extended by (3) is $x' = (0.3994, 0.3871, 0.2945, 0.2716, 1.0000, 0.2140)^t$ with the LP-relaxation value equal to 1297. This example also illustrates that the LBP is more difficult than the KP when it comes to the structure of the optimal LP-solution.

Furthermore, coefficients of the covering constraints (2) and (3) can be down-lifted as shown in the following Proposition. Let (for a fixed $i \in I$)

$$\tilde{C} = \begin{cases} C_i + 1, & i \leq i_c \\ C_c + 1, & \text{otherwise} \end{cases}$$

Proposition 3.2. *For a given $i \in I$, coefficients of the associated covering inequalities (2) can be down-lifted to $\sum_{k \in I} \alpha_k x_k \geq \tilde{C}$ where*

$$\alpha_k := \begin{cases} \min\{w_c, \tilde{C}\}, & k = i \\ \min\{w_k, \tilde{C}\}, & \text{otherwise} \end{cases} \quad \forall k \in I$$

Similarly, coefficients of the constraint (3) can be down-lifted as $\alpha_k = \min\{w_k, \tilde{C}\}$, for all $k \in I$.

Proof. We only show the first part of the proof. For a given $i \in I$, let us rewrite the covering constraint (2) as:

$$\sum_{j \in I, j \neq i} w_j x_j + \tilde{C} x_i \geq \tilde{C}$$

Lifting down from w_k to $\min\{w_k, \tilde{C}\}$ follows by standard arguments, given that we are dealing with binary decision variables. We can also down-lift the coefficient next to the variable x_i from $\tilde{C}$ to w_c (assuming $w_c < \tilde{C}$), which follows from the following arguments. If $x_i = 0$, the whole constraint makes sure that the capacity of the solution is at least $\tilde{C}$. On the other hand, if

$x_i = 1$, by Property (1.8), we know that the following inequality holds:

$$\sum_{j \in I, j \neq i} w_j x_j + w_i \geq C - w_c + 1.$$

The latter inequality can also be rewritten as $\sum_{j \in I, j \neq i} w_j x_j + w_c \geq \tilde{C}$. Hence, by reordering the coefficients, we end up with $\alpha_i = w_c$. $\square$ $\square$

Observe that after lifting, however, the covering inequality (2) associated to i_c and the global covering inequality (3) become identical.

3.1.2 Formulation (CP₁)

A constraint programming counterpart of the given ILP formulation is derived by following the problem definition. The set of chosen items cannot exceed the given capacity (see constraint (4)), and every item which is left outside, if added to the solution, exceeds C (see constraint (5)). The same set of x of binary variables is used to indicate selected items.

$$(CP_1) \quad \min_{x \in \{0,1\}^{|I|}} \sum_{i \in I} p_i x_i$$

$$\text{Pack}_{\leq}(w, x, C) \tag{4}$$

$$\text{if } (x_i = 0) \text{ then } \text{Pack}_{\geq}(w, x, C_i + 1) \quad \forall i \in I \tag{5}$$

In constraint (4), a bin-packing constraint $\text{Pack}_{\leq}(w, x, C)$ of size one (i.e., a knapsack constraint) is used to ensure that the selected items do not exceed the capacity. In our notation, $\text{Pack}_{\leq}(w, x, C)$ means that the set of items whose weights are given by a vector w , need to be packed into a knapsack of capacity C . The constraint is usually implemented by requiring that the items are placed in two bins, one of them having the capacity of C (and corresponding to items i such that $x_i = 1$), and the other of capacity $W - C$, containing the remaining items. Note that efficient constraint propagation techniques for bin-packing type of constraints exist, see, e.g., (Shaw, 2004; Trick, 2003). The remaining set of constraints (5) are a direct translation of inequalities (2) into the CP language. They impose that, for any item i , which is not selected, it must hold that the item cannot be added to the schedule without violating the capacity constraint. The latter is ensured with constraint $\text{Pack}_{\geq}(w, x, C_i + 1)$. Again, by complementing the x variables, $\text{Pack}_{\geq}(w, x, C + 1 - w_i)$ can be modeled as a bin-packing constraint. Note that this constraint can be down-lifted to $\text{Pack}_{\geq}(w \setminus w_i, x \setminus x_i, C_i + 1)$.

Model (CP₁) has $|I| + 1$ constraints. The size of the model can be improved by exploiting Proposition 1.8, i.e., the size of any feasible solution is bounded below by $C_c + 1$. Since $C_i \leq C_c$ for all $i \geq i_c$, we do not need to impose (5) for these items, but replace it with a single constraint

$$\text{Pack}_{\geq}(w, x, C_c + 1)$$

3.1.3 Hybridization and Branch-and-Cut

One could easily hybridize the (ILP₁) with (CP₁) by replacing the capacity inequalities (2) by the indicator constraints (5).

Concerning the implementation of the (ILP₁), there are two possibilities. The model, involving $i_c + 1$ constraints, can be given as-is (i.e., as a compact model) to a black-box MIP solver. Alternatively, one may start with a simple knapsack constraint (1) and insert the capacity inequalities (2) “on the fly” inside of the branch-and-bound tree, only when these inequalities are violated by the current LP-solution. This procedure, known as branch-and-cut, may be advantageous over the compact model, in particular if the number of items, and respectively the index of the critical item, are very large.

Both observations also hold for the remaining formulations presented below.

3.2 A Second (and Sparser) Formulation

To motivate this new formulation, let us first focus on the capacity inequality for $i = 1$:

$$\sum_{j \in I, j \neq 1} w_j x_j \geq (C_1 + 1)(1 - x_1).$$

Obviously, if $x_1 = 1$, this constraint is not binding, but for $x_1 = 0$, it will dominate the remaining covering constraints (2) of the previous formulation (ILP₁) (since the items are ordered in non-decreasing order according to w_i , which implies $C_1 \geq C_i$, for all $i > 1$).

Similarly, if $x_1 = 1$, and $x_2 = 0$, the constraint $\sum_{j \in I, j \neq 2} w_j x_j \geq (C_2 + 1)(1 - x_2)$ of the previous model will be binding. Hence, for $x_1 = 1$, and $x_2 = 0$, the latter constraint can be replaced by a stronger inequality:

$$\sum_{j \in I, j > 2} w_j x_j + w_1(1 - x_2) \geq (C_2 + 1)(1 - x_2),$$

which also remains valid for $x_2 = 1$ (since it is not binding in this case). One easily observes that we can continue deriving valid inequalities in this fashion, until we reach the critical item. We finally derive the following lifted model for the LBP:

$$\begin{aligned}
 (\text{ILP}_2) \quad & \min_{x \in \{0,1\}^{|I|}} \sum_{i \in I} p_i x_i \\
 & \sum_{i \in I} w_i x_i \leq C \\
 & \sum_{j > i} w_j x_j \geq (C + 1 - \sum_{j \leq i} w_j)(1 - x_i) \quad \forall i \in I, i < i_c \quad (6)
 \end{aligned}$$

The new covering constraints (6) (that we will refer to as *sparser covering constraints*) state that, if the item $i \in I$, $i < i_c$ is the smallest item left outside of the solution, the remaining capacity to be filled by the items j , $j > i$ has to be at least $C + 1 - \sum_{j \leq i} w_j$. We observe that covering constraints (6) for $i \geq i_c$ are redundant, since, by definition, in that case we have

$$C + 1 - \sum_{j \leq i} w_j < 0.$$

Proposition 3.3. *The model (ILP₂) is a valid formulation for the LBP.*

Proof. To show the validity of this model, we need to prove that a) no feasible solutions are cut off by this model, and b) all knapsack solutions that are not maximal are not feasible for this model.

- a) Assume that $S \subset I$ is a feasible LBP solution that is cut off by our model. Then, there exists a covering constraint (6) associated to an item $\tilde{i}$ such that S violates that constraint. Obviously, $\tilde{i} \notin S$, and let $\tilde{i} < i_c$ be the item with the smallest index such that constraint (6) is violated. Let x^* be the characteristic vector associated to S . So we have:

$$\sum_{j > \tilde{i}} w_j x_j^* + \sum_{j < \tilde{i}} w_j + w_{\tilde{i}} \leq C$$

This however contradicts the feasibility of S , since the latter inequality says that even if all items from $\{1, \dots, \tilde{i} - 1\}$ belong to S , S would not be a maximal knapsack solution, since by adding the item $\tilde{i}$ to it, we would still end up with a feasible packing of the knapsack.

- b) Let us now assume a subset $S \subset I$ is a feasible solution to our model such that it is a feasible knapsack packing, but not maximal, i.e., it can be extended by at least one more item without violating the capacity constraint. Let x^* be the characteristic vector of S . Let k be the index of the smallest item that does not belong to S . If $k > i_c$, then by definition of i_c , no further items can be added to S , so S is maximal.

Assume now that there exists an item that can be added to S so that S still remains a feasible packing. If such an item exists, then this certainly holds for the smallest item not in S as well. Since S is a feasible solution, it also satisfies the covering constraint (6) associated to k :

$$\sum_{j > k} w_j x_j^* \geq (C + 1 - \sum_{j \leq k} w_j)(1 - x_k^*)$$

i.e.

$$\sum_{j > k} w_j x_j^* \geq C + 1 - \sum_{j \leq k} w_j$$

Since $x_j^* = 1$ for all $j < k$, the latter inequality can also be rewritten as:

$$\sum_{j > k} w_j x_j^* + \sum_{j < k} w_j x_j^* + w_k \geq C + 1$$

which implies that adding the item k into the solution S would exceed the available knapsack capacity, which is a contradiction.

□

□

In a similar fashion as above, we can down-lift some of the coefficients associated to the covering

constraints (6).

Proposition 3.4. *For a given $i \in I$, let $\bar{C}_i = C + 1 - \sum_{j \leq i} w_j$. Coefficients of the associated capacity inequality (6) can be down-lifted to $\sum_{k \in I, k > i} \alpha_k x_k + \bar{C}_i x_i \geq \bar{C}_i$ where*

$$\alpha_k := \begin{cases} \min\{w_k, \bar{C}_i\}, & k > i \\ \min\{w_c, \bar{C}_i\}, & k = i \end{cases} \quad \forall k \geq i$$

A potential advantage of (ILP₂) over (ILP₁) is the sparsity of the covering constraints (6). Since less variables are involved, and even more, the constraint matrix has a triangular structure, we expect that these properties will be effectively exploited by ILP solvers.

3.2.1 Formulation (CP₂)

A CP counterpart of the (ILP₂) can be easily derived using the packing and the indicator constraint. In the formulation (CP₁), one has to replace (5) by:

$$\text{if } (x_i = 0) \text{ then Pack}_{\geq}((w)_{i+1}^n, (x)_{i+1}^n, C + 1 - \sum_{j \leq i} w_j) \quad \forall i < i_c \quad (7)$$

3.3 A Third (and Extended) Formulation

Recall that for any knapsack solution to be feasible for the LBP it is sufficient that adding the weight of the smallest item left out of the knapsack already violates the capacity C . Our next model encodes this information by extending the previous model with a non-negative continuous variable z modeling the weight of a smallest item that is left out of the solution.

$$\begin{aligned} \text{(ILP}_3\text{)} \quad & \min_{x \in \{0,1\}^{|I|}, z \geq 0} \sum_{i \in I} p_i x_i \\ & \sum_{i \in I} w_i x_i \leq C \end{aligned} \quad (8)$$

$$\sum_{i \in I} w_i x_i + z \geq C + 1 \quad (9)$$

$$z \leq w_c - (w_c - w_i)(1 - x_i) \quad \forall i \in I, i \leq i_c \quad (10)$$

This formulation contains only a single additional variable, but significantly simplifies the structure of the constraint matrix. Besides the packing constraint (8) and the covering constraint (9), the remaining matrix has a diagonal structure, plus one column of ones (corresponding to the variable z). Therefore, it is expected that the ILP solvers can be even more efficient when solving (ILP₃) than the previous formulations.

Validity of this model can be easily verified. Coupling constraints (10) make sure that $z \geq 0$ is smaller than the weight of the smallest item not included in the solution. Due to the covering

constraint (9) and the minimization objective function (with $p_i \geq 0$), it follows that z be exactly the same as the weight of the smallest item left out of the solution. For the same reasons as above (cf. Proposition 1.8), it is not necessary to impose constraints (10) for $i > i_c$.

3.3.1 Formulation (CP₃)

CP counterpart (CP₃) for the latter ILP model is also obtained with the help of an additional integer variable z .

$$\begin{aligned}
 (CP_3) \quad & \min_{x \in \{0,1\}^{|I|}, z \in \mathbb{N}} \sum_{i \in I} p_i x_i \\
 & \text{Pack}_{\leq}(w, x, C) \\
 & \text{Pack}_{\geq}(w, x, C + 1 - z) \tag{11} \\
 & z = \min\{w_i \mid i \in I, x_i = 0, i \leq i_c\} \tag{12}
 \end{aligned}$$

The set of constraints (5) (respectively, (7)) is replaced by the combination of constraints (11) and (12). Constraint (12) ensures that z is the weight of the smallest item not selected, and constraint (11) makes sure, that this smallest item can not be added to the selected items without violating the capacity constraint.

4. Greedy Heuristics

To complete our study, and to verify the difficulty of the studied benchmark instances, we have also implemented a greedy heuristic. Greedy heuristics are the most classical tool for constructing an initial feasible solution to knapsack-related problems. The approach consists of two basic steps: (i) examining the items according to a pre-specified order, and, at each iteration, (ii) adding the current item to the solution iff its weight does not exceed the current residual capacity.

The greedy algorithm for the traditional 0/1 KP sorts the items according to non-increasing p_j/w_j values. For the LBP, since the sum of profits needs to be minimized, different orderings need to be considered. We have tested the following six sorting criteria for creating feasible solutions:

$$1/p_j; \quad 1/w_j; \quad w_j/p_j; \quad 1/(p_j * w_j); \quad 1/(p_j + w_j); \quad p_j/w_j. \tag{13}$$

Note that in the greedy approximation algorithms of [Gourvés et al. \(2013\)](#) for the **time-spent** variant, the items are ordered non-decreasing according to p_i (which is equivalent to our first variant). Finally, since greedy heuristics are typically executed very fast, we also propose to combine all of them, and return the best obtained solution. This latter approach we call *greedy-comb*.

5. Computational Study and Conclusion

5.0.1 Benchmark Instances

We used the well-known instance generator of (Pisinger, 2014) for the classical 0/1 KP problem to randomly generate instances. Such instances are the classical instances used in the literature to test KP algorithms. Nine different classes of instances are obtained, see Martello et al. (1999) for more details.

For each instance class and value of $\bar{R} \in \{1000, 10000\}$, we generated 27 LBP instances by considering all combinations of (i) number of items $n \in \{10, 20, 30, 40, 50, 100, 500, 1000, 2000\}$; (ii) capacity $C \in \{\lfloor 0.25W \rfloor, \lfloor 0.50W \rfloor, \lfloor 0.75W \rfloor\}$ (and C increased by 1, if even, for classes 7 and 8); thus obtaining 486 LBP instances. In the next section we present the outcome of our computational study.

5.0.2 Results

All algorithms were coded in C++ and the experiments were performed on a cluster of computers, each consisting of 20 cores (2.3 GHz) and with 64GB RAM available for 20 cores. As ILP and CP solver we used Cplex 12.6 (single thread mode). Default time limit of 600 seconds was imposed to all runs.

We first compare the quality of solutions obtained by the proposed greedy heuristic, with respect to different selection criteria. Table 1 reports percentage gaps to the optimal (or best known solutions) averaged over all instances for all six selection variants presented above, and for the greedy-comb approach. We observe that among the single selection criteria, the one based on

Table 1: Average percentage gaps from best known solutions by different heuristic algorithms.

Algorithm	Class									avg
	1	2	3	4	5	6	7	8	9	
Greedy heuristics										
greedy[1/ p_j]	29.25	7.53	11.83	1.55	11.78	2.23	2.24	11.76	2.94	9.01
greedy[1/ w_j]	66.85	9.08	11.83	1.55	11.78	2.23	2.24	11.76	55.35	19.19
greedy[w_j/p_j]	6.71	2.16	1.92	1.55	2.11	2.23	2.24	1.85	2.94	2.63
greedy[1/($p_j * w_j$)]	56.20	8.35	11.83	1.55	11.78	2.23	2.24	11.76	2.94	12.10
greedy[1/($p_j + w_j$)]	56.35	8.39	11.83	1.55	11.78	2.23	2.24	11.76	4.20	12.26
greedy[p_j/w_j]	71.34	19.67	11.83	22.03	11.78	2.23	2.24	11.76	68.07	24.55
greedy-comb	6.71	1.14	1.03	1.55	1.07	2.23	2.24	0.96	2.82	2.19

sorting the items according to w_j/p_j performs the best, which is also intuitive, since in this case we prefer items with low profit and high weight. It is worth mentioning that the greedy-comb approach succeeds to improve the quality of solutions obtained by the best greedy approach, for several classes of instances, for which it seems that the other selection criteria perform better. Even for the largest instances, the greedy-comb takes less than one second, which makes it the method of choice for determining high-quality upper bounds.

The quality of lower bounds of the proposed ILP formulations is reported in Table 2. Lifted formulations (ILP_{*i*}) are denoted by (ILP_{*i*}^{*l*}) for $i = 1, 2$. Table 2 shows that lifting significantly reduces the LP-gap for both (ILP₁) and (ILP₂), and that (ILP₃) provides very strong lower

Table 2: Average percentage gaps from best known solutions by different linear programming relaxations.

Algorithm	Class									
	1	2	3	4	5	6	7	8	9	avg
Linear Relaxation										
ILP ₁	20.11	41.52	40.09	45.62	39.90	45.35	45.35	40.17	22.98	37.90
ILP ₁ ^l	13.67	3.19	2.75	2.52	2.68	2.92	2.93	2.84	6.64	4.46
ILP ₂	26.20	43.82	41.37	45.90	41.14	46.61	46.62	41.44	43.71	41.87
ILP ₂ ^l	13.75	4.30	3.28	2.46	3.25	2.64	2.64	3.38	6.64	4.71
ILP ₃	13.67	3.19	2.79	2.52	2.68	2.95	2.96	2.88	6.64	4.47

bounds, comparable to those obtained after lifting the first two formulations. Looking at different classes of instances, we notice that the first class (uncorrelated weights and profits) is by far the most difficult one, with average LP-gaps of more than 13% for all ILP formulations.

Finally, Table 3 provides a comparison of running times of three basic ILP formulations, (ILP₁^l), (ILP₂^l), two ILP+CP hybrids (in which indicator constraints are used), a branch-and-cut implementation of (ILP₁), constraint programming formulation (CP₃), and the DP algorithm. The first half of the table shows the average computing times over all instances, sorted according

Table 3: Comparison between the different exact methods proposed.

Items	ILP ₁				ILP ₂			ILP ₃	CP ₃	DP
	ILP ₁	ILP ₁ ^l	B&C	ILP ₁ + CP	ILP ₂	ILP ₂ ^l	ILP ₂ + CP			
Avg t[sec.s]										
10	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	33.3
20	0.1	0.0	0.0	0.1	0.1	0.0	0.1	0.0	6.1	55.6
30	0.3	0.1	0.1	0.5	0.2	0.1	0.3	0.1	432.3	55.6
40	0.6	0.2	0.2	1.3	0.5	0.2	0.7	0.2	555.8	66.7
50	1.1	0.2	0.7	2.7	0.8	0.3	1.3	0.2	557.6	66.7
100	45.6	1.9	30.5	61.2	34.5	26.7	31.6	1.2	600.0	67.2
500	315.9	59.9	341.6	362.0	357.4	169.3	402.9	56.0	600.0	257.3
1000	476.6	60.7	367.5	542.9	531.2	185.9	555.8	34.3	600.0	377.1
2000	552.8	152.3	445.7	567.9	584.1	230.5	578.8	61.5	600.0	555.8
# of TL										
10	0	0	0	0	0	0	0	0	0	3
20	0	0	0	0	0	0	0	0	0	5
30	0	0	0	0	0	0	0	0	36	5
40	0	0	0	0	0	0	0	0	50	6
50	0	0	0	0	0	0	0	0	50	6
100	1	0	1	3	2	1	1	0	54	6
500	22	5	27	24	25	15	30	5	54	22
1000	32	3	31	45	44	16	48	3	54	30
2000	48	3	37	51	52	18	51	5	54	46

to the number of items. The second half reports the number of unsolved instances per each group (out of 54). Results for (CP₁) and (CP₂) are not reported, as they appear even slower than (CP₃). We notice that the best performing exact approaches are (ILP₁^l) and (ILP₃) which we explain by a fact: very tight lower bounds. Among the whole benchmark set of 486 instances, less than 20 remain unsolved within 10 minutes by running these approaches. On the contrary, for CP and DP algorithms, instances with ≥ 100 already appear very difficult: none of them could be solved to optimality within the time-limit by CP, and less than half of them with ≥ 500 items could not be solved by the DP. The DP did especially struggle to solve instance class 9, where the value of the capacity is high. Surprisingly, running the B&C implementation of (ILP₁^l) is slower than solving the compact model, which may be explained by the strength

of general-purpose CPLEX cuts (like knapsack-cover inequalities, MIR or 0-1/2 cuts) that are better exploited if complete information on the structure of the solution is given to the solver, rather than when this information is provided “on the fly”.

5.0.3 Conclusions and Future Work

This article considers integrative ILP and CP approaches for solving the LBP problem to optimality. As pointed out at the beginning of this article, one could alternatively solve the problem by resolving a linear number of iterations of a knapsack problem with a lower and upper bound on its capacity. We are currently working on a faster dynamic programming algorithm and alternative formulations that might be competitive against the approaches presented in this paper. Furthermore, we consider the CP formulations presented in this article as a first attempt to model the problem using constraint programming and hope to draw attention of the CP community to this interesting problem for which we believe more efficient approaches could be developed, by hybridizing ILP, CP and DP techniques. In particular, an adaptation of a DP algorithm could be used for propagation (as it was proposed in [Trick \(2003\)](#) for the knapsack problem).

“Lazy” reformulations of other standard combinatorial optimization problems (such as, bin packing) have been recently proposed (see, e.g. [Boyar et al. \(2006\)](#)), but only approximation algorithms have been studied so far. Studying lazy packing or scheduling problems is a particularly interesting field of research for both the IP and the CP community, since it gives rise to a rich set of new solution properties, valid inequalities, and studies of the opposite side of the polytope, as the optimization is driven in the opposite direction when compared to their standard (non-lazy) counterparts. Our computational study showed that the LBP is more difficult than the KP: for the latter, instances with several thousands of items can be easily solved, while for the LBP, a few thousands items make the problem difficult (using ILP formulations). Finally exact methods for the LBP are especially important since, as the lazy counterpart of the KP, the LBP is a basic problem which appears as a subproblem in many lazy reformulations. Effective methods for the LBP can then lead to computational speed-up for a large set of lazy problems.

Acknowledgements

The authors want to thank to Alberto Ceselli, Silvano Martello, Michele Monaci and Ulrich Pferschy for useful discussions. The research of M. Sinnl was supported by the Austrian Research Fund (FWF, Project P26755-N19).

References

- Esther M. Arkin, Michael A. Bender, Joseph S.B. Mitchell, and Steven S. Skiena. The lazy bureaucrat scheduling problem. *Information and Computation*, 184(1):129–146, 2003.
- Joan Boyar, Leah Epstein, Lene M. Favrholdt, Jens S. Kohrt, Kim S. Larsen, Morten M. Pedersen, and Sanne Wøhlk. The maximum resource bin packing problem. *Theoretical Computer Science*, 362(1–3):127–139, 2006.
- Marco Casazza and Alberto Ceselli. Mathematical programming algorithms for bin packing problems with item fragmentation. *Computers & Operations Research*, 46(0):1–11, 2014.
- Ling Gai and Guochuan Zhang. On lazy bureaucrat scheduling with common deadlines. *Journal of Combinatorial Optimization*, 15(2):191–199, 2008.
- L. Gourvés, J. Monnot, and Aris T. Pagourtzis. The lazy bureaucrat problem with common arrivals and deadlines: Approximation and mechanism design. In Leszek Gasieniec and Frank Wolter, editors, *Fundamentals of Computation Theory*, volume 8070 of *Lecture Notes in Computer Science*, pages 171–182. Springer Berlin Heidelberg, 2013.
- S. Martello, D. Pisinger, and P. Toth. Dynamic programming and strong bounds for the 0-1 knapsack problem. *Management Science*, 45:414–424, 1999.
- David Pisinger. David Pisinger’s optimization codes, 2014. <http://www.diku.dk/~pisinger/codes.html>.
- Paul Shaw. A constraint for bin packing. In Mark Wallace, editor, *Principles and Practice of Constraint Programming – CP 2004*, volume 3258 of *Lecture Notes in Computer Science*, pages 648–662. Springer Berlin Heidelberg, 2004.
- Michael A. Trick. A dynamic programming approach for consistency and propagation for knapsack constraints. *Annals of Operations Research*, 118(1–4):73–84, 2003.